

# Dynamic Sets

Dynamic sets in Comp. Science change over time

↳ Some dynamic sets assume a unique attribute "key" of the object

↳ Operations on a set  $S$  include:

Search( $S, k$ ) :  $k$  : key

Insert( $S, x$ ) :  $x$  : object or pointer

Delete( $S, x$ )

Minimum( $S$ )/Maximum( $S$ ) :

Successor( $S, x$ ) / Predecessor( $S, x$ )

on an  
ordered set

↳ A dynamic set that supports these operations is called a Dictionary

Arrays — Contiguous memory for storing elements of a set

↳ Fixed length

↳ Contiguous

↳ Same type of object

Indexing an array

↳ Row major (a.k.a. Scan order)  
(Left → Right) — (Top → Bottom)

↳ Column major  
(Top → Bottom) — (Left → Right)

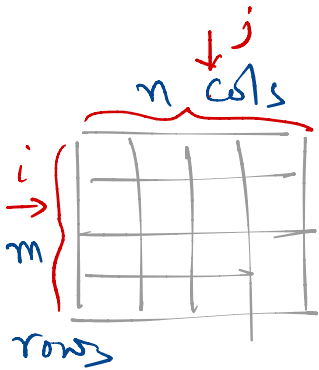
Given starting address a, each element  
( $s=0$  or  $s=1$ )  
occupying b bytes in an s-based indexing,  
the  $i^{\text{th}}$  element has byte address  $a + b(i-s)$

2D arrays : Allocate an  $m \times n$  array in C:

$\text{int} * \text{mat} = (\text{int} *) \text{malloc}(\# \text{elements} * \text{size of each elem.})$

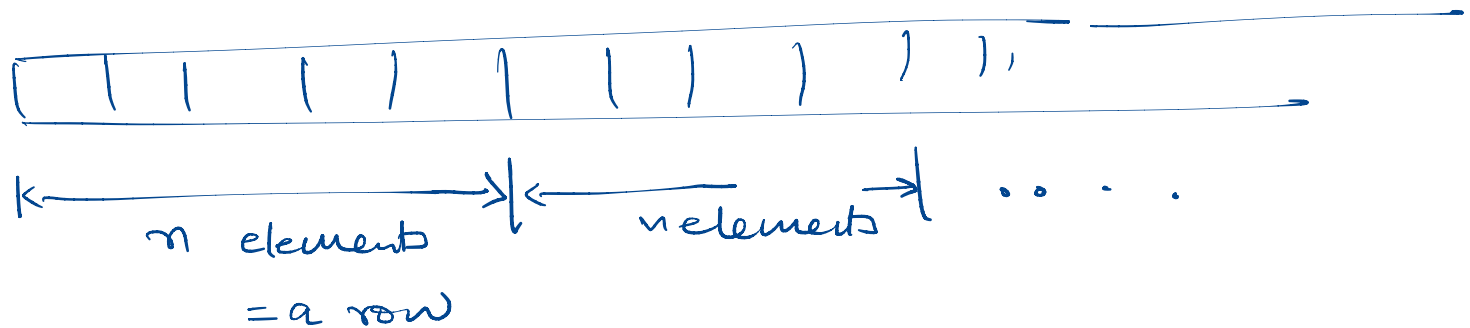
$= (\text{int} *) \text{malloc}(m * n * \text{sizeof}(\text{int}))$

Accessing an element at  $(i, j)$  (in a 0-based indexing)



$\text{mat}[i * n + j]$  or  $*(\text{mat} + i * n + j)$

row major



# Stacks

a dynamic set implementation s.t.  
(Last-in first-out)

↳ implements LIFO policy

↳ Insert operation is called PUSH

↳ Delete operation is called POP

— An array based implementation

↳ a stack of at most  $n$  elements  
with array  $A[1:n]$

↳ An attribute  $S.top$  → index of the  
recently inserted  
element

poping from  
empty stack ←  
⇒ "underflow"

↳  $S[1]$  is the bottom of the stack

↳ when  $S.top == 0$ , the stack is empty

↳ when  $S.top$  exceeds  $n$  ⇒ "stack overflow"



Stack - empty (s)

if  $s.top == 0$   
    return true  
else  
    return false

Push (s, x)

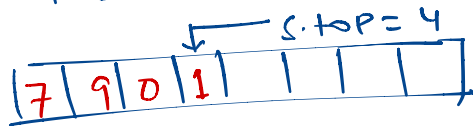
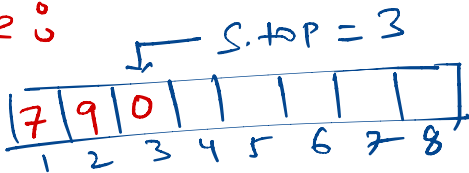
$s.top = s.top + 1$   
 $S[s.top] = x$

$O(1)$   
Todo: check for overflow!  
 $O(1)$

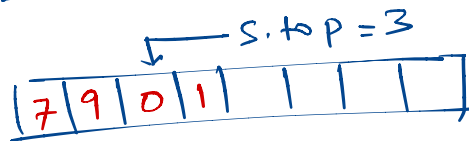
Pop (s)

if stack - empty (s)  
    Error "Underflow"  
else  
     $s.top = s.top - 1$   
    return  $S[s.top + 1]$

Example :



Push (s, 1)



Pop (s)

Queues : In a queue, the deleted element is at the head.

(First-in First-out)

- Implements FIFO policy
- Insert operation is called Enqueue
- Remove operation is called Dequeue
- Has a head & a tail
- Enqueue happens at the tail
- Dequeue happens at the head

→ Array based implementation

→ A queue of at most  $(n-1)$  elements using

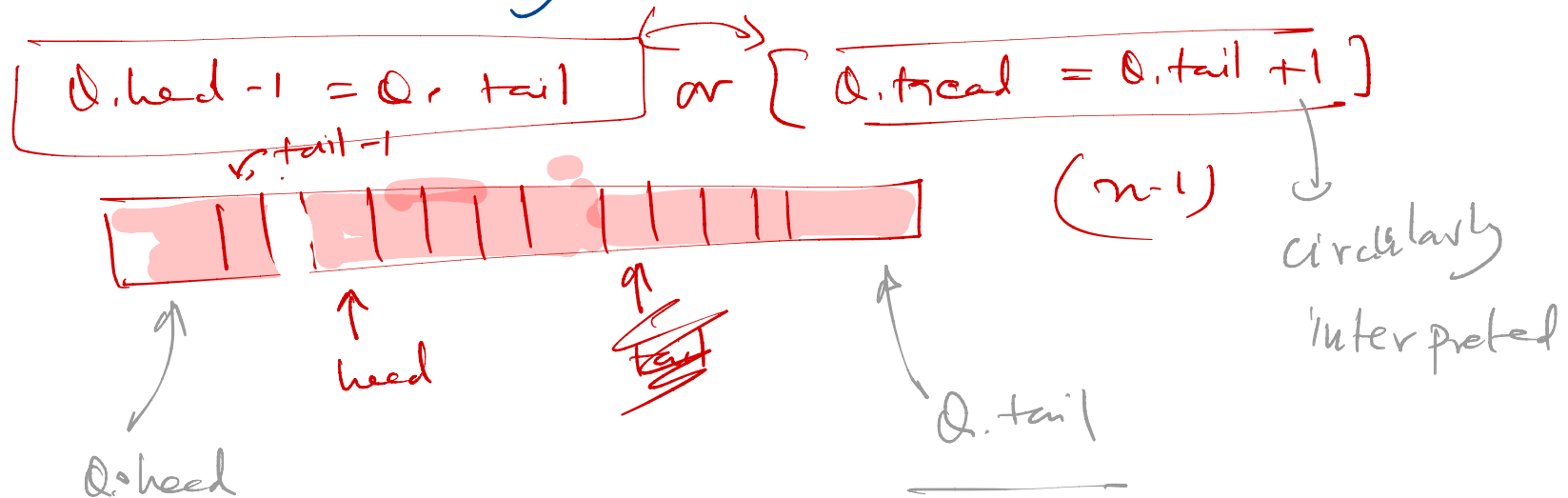
$Q[1:n]$

- Attributes :
  - $Q.head$  → index of head
  - $Q.tail$  → index of tail
- Elements reside in  $Q.head, Q.head+1, \dots$   
 $\dots, Q.tail-1$  (index of next insertion)

→ "wrap around" the array in a circular fashion. So, 1 follows  $n$ .

→  $Q$  is empty when  $Q.head == Q.tail$

→ Initially set  $Q.head = Q.tail = 1$



→ Dequeue from empty queue  $\Rightarrow$  "underflow"

→ Overflow :  $(Q.head == Q.tail + 1)$

OR

$(Q.head == 1 \text{ AND } Q.tail = Q.Size)$

Further Enqueue will result in "overflow"

Assume  $n = Q.length$

→ Allocate  $O[1:n]$  Array

---

Enqueue( $Q, x$ )

---

$Q[Q.tail] = x$

if  $Q.tail == Q.length$   
     $Q.tail = 1$

else  $Q.tail = Q.tail + 1$

---

---

Dequeue( $Q$ )

---

$x = Q[Q.head]$

if  $Q.head == Q.length$   
     $Q.head = 1$

else  $Q.head = Q.head + 1$

return  $x$

---

$O(1)$

Todo: Detect  
underflow  
and  
overflow

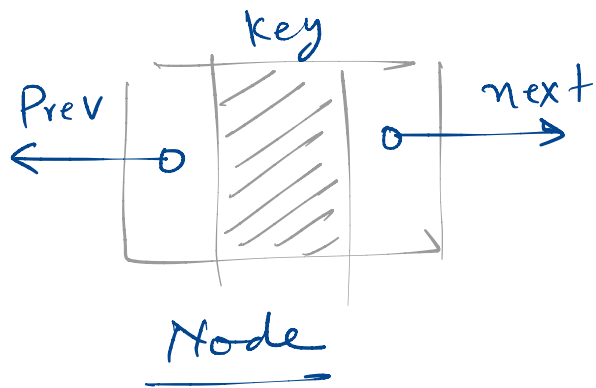
$O(1)$

Linked Lists :

## Linked Lists

- A data structure where objects are arranged in a linear order.
- The order is determined by a pointer in each object
- A singly linked list allows traversal in one direction
- A Doubly linked list allows bidirectional traversal

- A doubly linked list  $L$  is an object with attributes:



→ key : data

→ next : Pointer to the next object

→ prev : Pointer to the previous object.

→ Object may contain other  
satellite data  
(additional)

- For an element  $x$ ,

-  $x.next$  points to the successor of  $x$

-  $x.prev$  points to the predecessor of  $x$

- if  $x.prev == NIL \Rightarrow x$  has no predecessor  
 $\Rightarrow x$  is the Head or first element

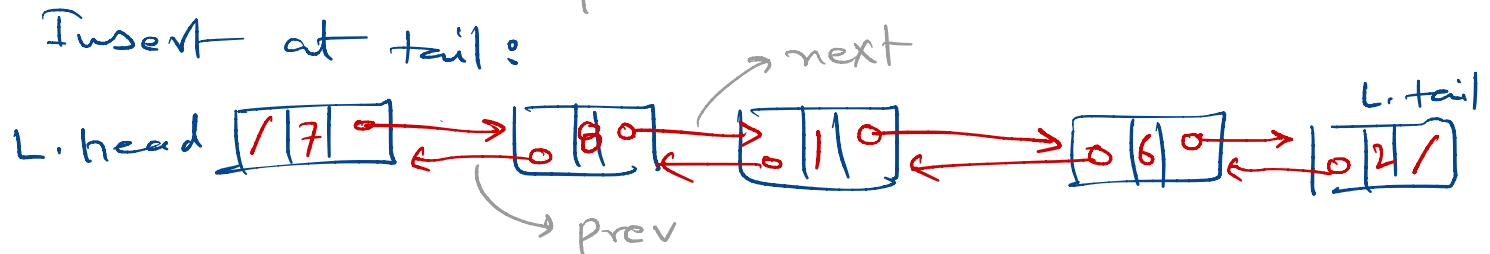
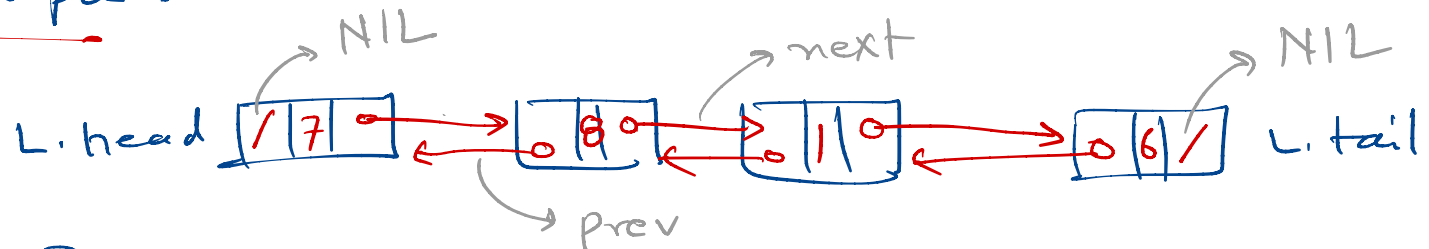
- if  $x.next == NIL \Rightarrow x$  has no successor  
 $\Rightarrow x$  is the Tail or last element

- Attribute : L.head  $\rightarrow$  points to the first object  
L.tail  $\rightarrow$  points to the last element

— A linked list could be

- singly or doubly
- sorted or unsorted
- circular or not

— Example :



→ Searching a linked list

→ find first element with given key  $k$  in  $L$

→ return NIL if not found

---

List-Search( $L, k$ )

---

$x = L_{\text{head}}$

while  $x \neq \text{NIL}$  and  $x.\text{key} \neq k$

$x = x.\text{next}$

return  $x$

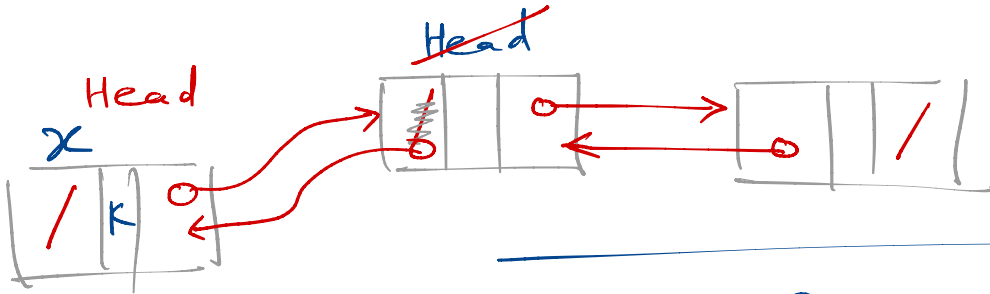
---

$\theta(n)$   
complexity



## Inserting into a linked list

Prepend: splices an element  $x$  (whose key has been set) onto the front of  $L$



---

List-Prepend( $L, x$ )

---

$x \cdot \text{next} = L \cdot \text{head}$

if  $L \cdot \text{head} \neq \text{NIL}$

$L \cdot \text{head} \cdot \text{prev} = x$

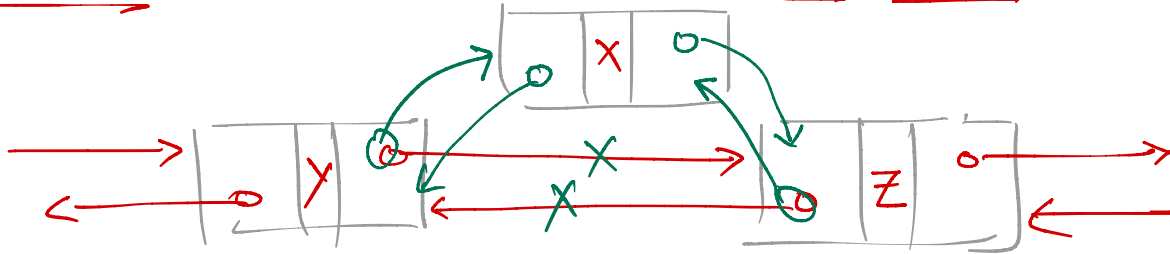
$L \cdot \text{head} = x$

$x \cdot \text{prev} = \text{NIL}$

---

Todo:  
Set tail  
Pointer  
appropriately

Insert : Insert element x after element y.



---

List-Insert (x, y)

---

$x.\text{next} = y.\text{next}$

$x.\text{prev} = y$

if  $y.\text{next} \neq \text{NIL}$

$y.\text{next}.\text{prev} = x$

$y.\text{next} = x$

---

Deleting from a linked list L

- removes an element X from L
- Splices X out of list by updating pointers
- First call List-Search to retrieve X for a given key.

---

List-Delete (L, X)

---

If  $X.\text{prev} \neq \text{NIL}$

$X.\text{prev}.\text{next} = X.\text{next}$

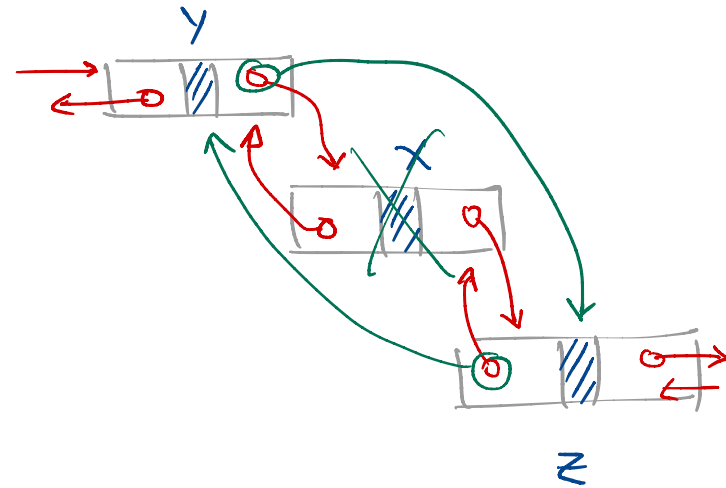
else

$L.\text{head} = X.\text{next}$  //  $\Rightarrow$  X was head

if  $X.\text{next} \neq \text{NIL}$

$X.\text{next}.\text{prev} = X.\text{prev}$

---



→ Reversing a doubly linked list

→ swap prev & next pointers

→ Assign last node to head

---

List\_Reverse(L)

---

If (L.head.next == NIL)

return // No need to reverse a single-element list

x = L.head

while (x ≠ NIL)

tmp = x.prev

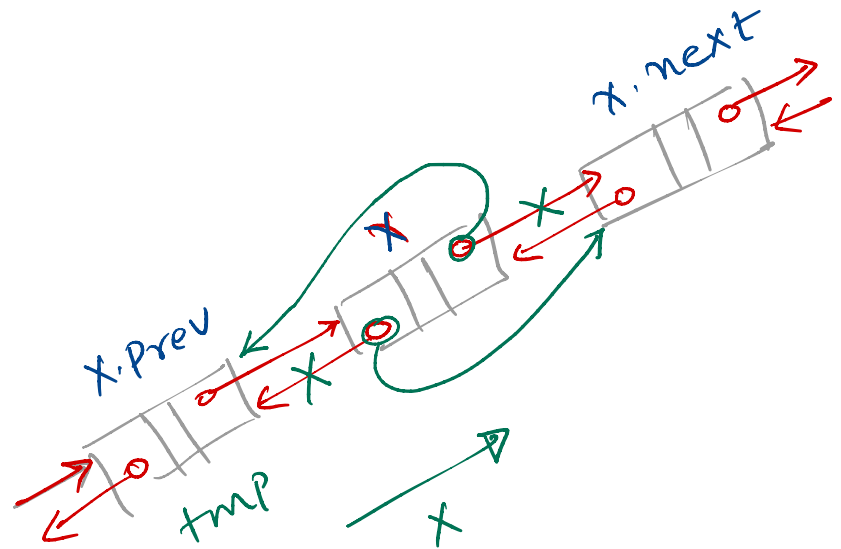
x.prev = x.next

x.next = tmp

x = x.prev

L.head = tmp.next

---



→ Join of two lists

Given lists  $L_1$  &  $L_2$  return a concatenation of  
 $L_1$  &  $L_2$

---

List-Join ( $L_1, L_2$ )

---

$L_1 \cdot \text{tail} \cdot \text{next} = L_2 \cdot \text{head}$

$L_2 \cdot \text{head} \cdot \text{prev} = L_1 \cdot \text{tail}$

return  $L_1 \cdot \text{head}$

---

Implement a queue using singly list

→ keep a pointer to head as well as tail in the singly list

---

Enqueue (Q, x) // insert after tail

---

n = node(x) // create a new node

Q.tail.next = n // insert at tail

n.next = NIL

Q.tail = n

---

---

Dequeue (Q)

---

x = Q.head

Q.head = Q.head.next

return x

---

And deallocate memory for the returned obj.